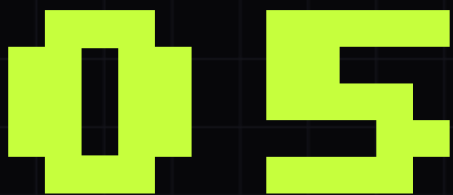


MODULE 05 OF 09



Client & Server

YOU'LL SHIP

A secure pipeline for player actions in Coin Rush.

- 5.1 How the client-server model works
- 5.2 RemoteEvents and RemoteFunctions
- 5.3 Never trust the client
- 5.4 Hardening your game against exploiters

This is the module that separates hobby projects from real games. Every Roblox game is multiplayer, which means code runs in two places at once. Get this right and your game is fair, secure and bug-free. Get it wrong and exploiters will give themselves a million coins on day one.

5.1 How the client-server model works

When someone plays your game:

- **The server** is a Roblox computer in a data center. There's one per game instance. It's the source of truth: coins, health, rounds, purchases.
- **The client** is each player's device. It renders the world, plays sounds, shows UI and reads input.

Replication keeps them in sync. When the server changes something in Workspace, like moving a part or spawning a coin, every client sees it. When a client changes something, it usually stays on that client only. That's intentional: it stops one player from changing the world for everyone.

Lives on	Examples
Server only	ServerScriptService, ServerStorage, DataStores, purchases
Client only	PlayerGui, the camera, input, LocalScripts
Both	Workspace, ReplicatedStorage

One notable exception: a player's **own character** movement is controlled by their client (so movement feels instant), then replicated. That's why speed hacks exist, and why the server must sanity-check.

5.2 RemoteEvents and RemoteFunctions

When the client and server need to talk, they use remotes. Create them in **ReplicatedStorage** so both sides can see them.

RemoteEvent: a one-way message.

```
-- ReplicatedStorage/Remotes/BuyUpgrade (RemoteEvent)

-- Client (LocalScript): ask the server to buy something
local ReplicatedStorage = game:GetService("ReplicatedStorage")
local BuyUpgrade = ReplicatedStorage.Remotes.BuyUpgrade

shopButton.Activated:Connect(function()
    BuyUpgrade:FireServer("Speed")
end)
```

```
-- Server (Script in ServerScriptService): handle the request
local ReplicatedStorage = game:GetService("ReplicatedStorage")
local BuyUpgrade = ReplicatedStorage.Remotes.BuyUpgrade

BuyUpgrade.OnServerEvent:Connect(function(player, upgradeName)
    print(player.Name, "wants to buy", upgradeName)
end)
```

Notice that the server receives `player` as the first argument **automatically**. The client can't fake who they are.

The server can message clients too:

```
-- Server → one player
Notify:FireClient(player, "Round starting!")

-- Server → everyone
Notify:FireAllClients("Round starting!")
```

RemoteFunction: a request that waits for an answer.

```
-- Client
local price = GetPrice:InvokeServer("Speed")

-- Server
GetPrice.OnServerInvoke = function(player, upgradeName)
    return GameConfig.UpgradePrices[upgradeName]
end
```

WATCH OUT Never use `InvokeClient` from the server. If the client never answers, the server script hangs forever.

5.3 Never trust the client

Here's the golden rule of Roblox development: **the client can send anything**. Exploiters can fire any `RemoteEvent`, with any arguments, as often as they like.

This code is a disaster:

```
-- ❌ DON'T: the client decides how many coins it gets
AddCoins.OnServerEvent:Connect(function(player, amount)
    player.leaderstats.Coins.Value += amount
end)
```

An exploiter just calls `AddCoins:FireServer(999999)`.

The fix is to send **intent**, not results. The client says *what it wants to do*; the server checks whether it's allowed and decides the outcome:

```
-- ✅ DO: the client asks, the server decides
BuyUpgrade.OnServerEvent:Connect(function(player, upgradeName)
    -- 1. Validate the type and the value
    if typeof(upgradeName) ~= "string" then return end
    local price = GameConfig.UpgradePrices[upgradeName]
    if not price then return end

    -- 2. Check the rules on the server
    local coins = player.leaderstats.Coins
    if coins.Value < price then return end
    if player:GetAttribute("Has" .. upgradeName) then return end

    -- 3. Apply the result on the server
    coins.Value -= price
    player:SetAttribute("Has" .. upgradeName, true)
end)
```

The same goes for collecting coins: the server should detect the touch and award the coin. The client should never say "I collected a coin".



Hardening your game against exploiters

A checklist to run on every remote:

1. **Validate types** with `typeof()`. Reject anything unexpected.
2. **Validate ranges**. Numbers can be negative, huge, `NaN` or `math.huge`. Clamp or reject.
3. **Check permissions and state**. Can this player do this *right now*? (Alive? In the round? Enough coins?)
4. **Rate-limit**. Ignore requests that come too fast:

```

local lastRequest: { [Player]: number } = {}

local function allowed(player: Player, cooldown: number): boolean
    local now = os.clock()
    if lastRequest[player] and now - lastRequest[player] < cooldown then
        return false
    end
    lastRequest[player] = now
    return true
end

game:GetService("Players").PlayerRemoving:Connect(function(player)
    lastRequest[player] = nil -- avoid memory leaks
end)

```

5. **Sanity-check movement.** For Coin Rush, when a player collects a coin, check they're actually close to it:

```

local function isNear(player: Player, part: BasePart, maxDistance: number): boolean
    local root = player.Character and player.Character:FindFirstChild("HumanoidRootPart")
    return root ~= nil and (root.Position - part.Position).Magnitude <= maxDistance
end

```

6. **Keep secrets on the server.** Don't put prices you enforce, loot tables, or admin lists in ReplicatedStorage and trust them from the client. Put server-only code and assets in ServerScriptService and ServerStorage.

TIP A good test: imagine the player's LocalScripts were rewritten by the worst person on the internet. Is your game still fair? If yes, you're done.

TRY IT

1. Create ReplicatedStorage/Remotes with a BuyUpgrade RemoteEvent and implement the secure handler above.
2. Add the rate limiter so a player can only attempt one purchase every 0.5 seconds.

Module recap

- ☐ You can explain what runs on the server vs. the client
- ☐ You can send messages both ways with RemoteEvents

- ☐ Every remote validates types, values, state and rate
- ☐ Clients send intent; the server decides results